

Apache Airflow

Exercise 2 — Daily Attendance Processor DAG

daily_attendance_processor

Name: Vaishali S

Program: Data Engineering | Batch: 2

Date: 2026-05-09

Topics Covered

Graph View • Task Instances • Run Details • Terminal Output

1. Exercise Overview

About This Exercise

This exercise demonstrates creating and running a daily attendance processor workflow DAG in Apache Airflow. The DAG reads student attendance data, counts totals, calculates the attendance percentage, lists absent students, and generates a daily report — all orchestrated as an 8-step Python pipeline.

1.1 DAG Summary

Property	Value
DAG Name	daily_attendance_processor
Schedule	Manual Trigger
Logical Date	2026-05-09 10:39:20
Start Date	2026-05-09 10:39:25
End Date	2026-05-09 10:39:38
Duration	00:00:12.957
Run Type	Manual
Triggering User	admin
DAG Version	v1
Task Instances	8 — all Success
Operator Used	PythonOperator (all 8 tasks)

1.2 Task Pipeline (8 Tasks)

The DAG runs 8 tasks in a linear sequence using PythonOperator:

- `create_attendance_file` — Creates `attendance.txt` with student name and status
- `read_attendance_file` — Reads and parses attendance records from the file
- `count_total_students` — Counts total number of student entries
- `count_present_students` — Counts students marked as Present
- `count_absent_students` — Counts students marked as Absent
- `calculate_attendance_percentage` — Computes attendance percentage
- `list_absent_students` — Extracts list of absent student names
- `generate_attendance_report` — Writes the final summary to `attendance_report.txt`

```
create_attendance_file >> read_attendance_file >> count_total_students >> count_present_students >>
count_absent_students >> calculate_attendance_percentage >> list_absent_students >>
generate_attendance_report
```

2. Screenshot 1 — DAG Graph View with Run Details

The Graph View shows all 8 tasks as green nodes indicating successful completion. The right panel displays the run metadata: logical date, run type, start/end dates, duration, and task instances list.

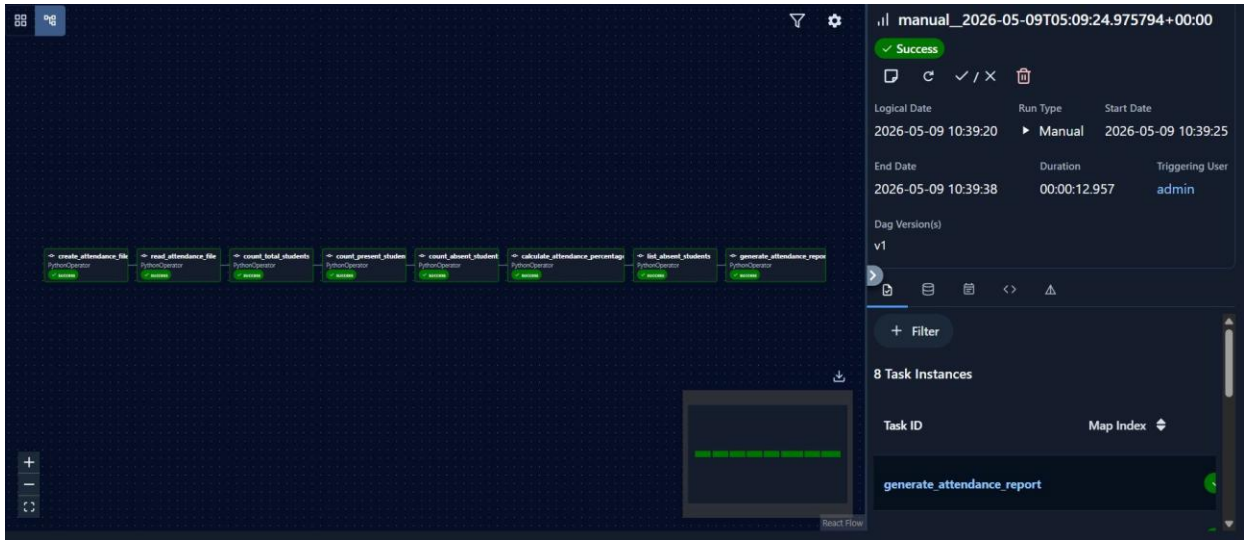


Figure 1: Graph View showing all 8 tasks with Success status and run details panel

Property	Value / Observation
Run ID	manual__2026-05-09T05:09:24.975794+00:00
Run Status	✓ Success
Logical Date	2026-05-09 10:39:20
Run Type	Manual
Start Date	2026-05-09 10:39:25
End Date	2026-05-09 10:39:38
Duration	00:00:12.957
Triggering User	admin
DAG Version	v1
Task Node Colour	Green — all 8 tasks completed successfully

2.1 Understanding the Graph View

In the Graph View, each coloured box represents a task. Green indicates success. The minimap (bottom-right) gives a bird's-eye view of the full pipeline. The right panel lists the last triggered task instance (generate_attendance_report) confirming all 8 task instances were created for this run.

3. Screenshot 2 — Task Instances Detail (Part 1)

The Task Instances tab provides a detailed breakdown of each task's execution: state, start time, try count, operator, and duration. All tasks completed on the first attempt with PythonOperator.

Task ID	Map Index	State	Start Date	Try Number	Operator	Duration
generate_attendance_report		Success	2026-05-09 10:39:37	1	PythonOperator	00:00:00.163
list_absent_students		Success	2026-05-09 10:39:36	1	PythonOperator	00:00:00.140
calculate_attendance_percent age		Success	2026-05-09 10:39:35	1	PythonOperator	00:00:00.179
count_absent_students		Success	2026-05-09 10:39:34	1	PythonOperator	00:00:00.168
count_present_students		Success	2026-05-09 10:39:33	1	PythonOperator	00:00:00.132

Figure 2: Task Instances tab showing generate_attendance_report, list_absent_students, calculate_attendance_percentage, count_absent_students, count_present_students

Task ID	Try	Start Date	Duration
generate_attendance_report	1	10:39:37	00:00:00.163
list_absent_students	1	10:39:36	00:00:00.140
calculate_attendance_percentage	1	10:39:35	00:00:00.179
count_absent_students	1	10:39:34	00:00:00.168
count_present_students	1	10:39:33	00:00:00.132

3.1 Task Execution Pattern

Tasks 4–8 (count_present_students through generate_attendance_report) each ran in under 0.2 seconds, showing efficient in-memory operations after the initial file creation. Each task ran exactly once (Try Number = 1), indicating zero retries — the pipeline ran clean on the first attempt.

4. Screenshot 3 — Task Instances Detail (Part 2)

This view shows the remaining tasks in the pipeline (count_total_students, read_attendance_file, create_attendance_file) completing the full picture of all 8 task executions.

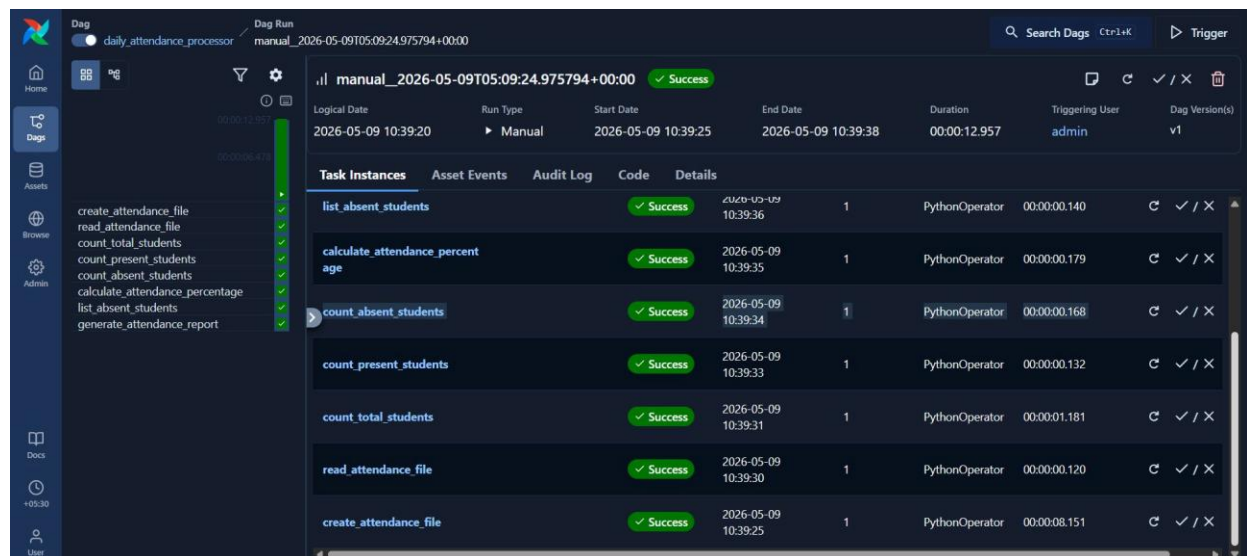


Figure 3: Task Instances showing `count_total_students`, `read_attendance_file`, and `create_attendance_file`

Task ID	Try	Start Date	Duration
count_total_students	1	10:39:31	00:00:01.181
read_attendance_file	1	10:39:30	00:00:00.120
create_attendance_file	1	10:39:25	00:00:08.151

4.1 Notable Observations

create_attendance_file — 8.151 seconds

This is the longest-running task because it is responsible for writing the `attendance.txt` file to disk. The delay reflects file I/O time and any initial container setup overhead.

count_total_students — 1.181 seconds

Slightly longer than other counting tasks, likely due to reading and iterating over all 10 student records. Still sub-second class for practical purposes.

All Tasks — Try Number = 1

Every single task ran on its first attempt with no retries, confirming the DAG code is correct and the container environment is stable.

4.2 Complete Task Instance Summary

Task ID	Try	Start Date	Duration	State
create_attendance_file	1	10:39:25	00:00:08.151	✓ Success
read_attendance_file	1	10:39:30	00:00:00.120	✓ Success
count_total_students	1	10:39:31	00:00:01.181	✓ Success
count_present_students	1	10:39:33	00:00:00.132	✓ Success
count_absent_students	1	10:39:34	00:00:00.168	✓ Success
calculate_attendance_percentage	1	10:39:35	00:00:00.179	✓ Success

list_absent_students	1	10:39:36	00:00:00.140	✓ Success
generate_attendance_report	1	10:39:37	00:00:00.163	✓ Success

5. Screenshot 4 — Terminal: Docker Exec & File Contents

This terminal output from inside the Docker container confirms the DAG correctly created both the input attendance file and the output attendance report in the /tmp directory.

```
root@ubuntu:~$ docker exec -it airflow bash
airflow@c3e149605791:/opt/airflow$ cat /tmp/attendance.txt
Aarav,Present
Priya,Present
Rahul,Absent
Sneha,Present
Kiran,Absent
Ananya,Present
Vikram,Present
Meera,Absent
Farhan,Present
Divya,Present
airflow@c3e149605791:/opt/airflow$ cat /tmp/attendance_report.txt
Daily Attendance Report
Total Students = 10
Present Students = 7
Absent Students = 3
Attendance Percentage = 70%
Status = Needs Improvement
airflow@c3e149605791:/opt/airflow$
```

Figure 4: Docker exec terminal showing attendance.txt and attendance_report.txt contents

Property	Value / Observation
Command	docker exec -it airflow bash
Input File	/tmp/attendance.txt
Student Records (10)	Aarav,Present Priya,Present Rahul,Absent Sneha,Present Kiran,Absent Ananya,Present Vikram,Present Meera,Absent Farhan,Present Divya,Present
Output File	/tmp/attendance_report.txt
Total Students	10
Present Students	7
Absent Students	3 (Rahul, Kiran, Meera)
Attendance Percentage	70%
Status	Needs Improvement (threshold not met)

5.1 Output Files Explained

attendance.txt — Input File

Created by the create_attendance_file task. Contains comma-separated name-status pairs (e.g. Aarav,Present) for all 10 students. Used by all downstream tasks for counting, percentage calculation, and report generation.

attendance_report.txt — Output File

Generated by generate_attendance_report. Contains: Daily Attendance Report / Total Students = 10 / Present Students = 7 / Absent Students = 3 / Attendance Percentage = 70% / Status = Needs Improvement. The status logic flags attendance below a defined threshold (e.g. 75%) as needing improvement.

5.2 Terminal Commands

```
$ docker exec -it airflow bash
$ cat /tmp/attendance.txt
Aarav,Present | Priya,Present | Rahul,Absent | Sneha,Present | Kiran,Absent
Ananya,Present | Vikram,Present | Meera,Absent | Farhan,Present |
Divya,Present
$ cat /tmp/attendance_report.txt
Daily Attendance Report
Total Students = 10 | Present Students = 7 | Absent Students = 3
Attendance Percentage = 70% | Status = Needs Improvement
```

6. Summary

This exercise successfully demonstrated a complete end-to-end attendance processing workflow in Apache Airflow. The daily_attendance_processor DAG read student records, computed attendance statistics, identified absent students, and produced a verified report — all with zero failures across all 8 tasks.

What Was Demonstrated	Outcome
Manual DAG Trigger	Triggered via UI — run completed in 12.957s
8-task linear pipeline	All tasks completed with PythonOperator
File I/O operations	attendance.txt created, read, report written
Attendance calculation	7 present / 3 absent out of 10 = 70%
Status logic	Status = Needs Improvement (below threshold)
Zero failures	0 failed tasks — all 8 ran on first attempt